

Nástroj pro hashtag integraci v MS Outlook

Hashtag Plugin for MS Outlook

Jakub Kubisz

Bakalářská práce

Vedoucí práce: Ing. Svatopluk Štolfa, Ph.D.

Ostrava, 2024

Zadání bakalářské práce

Student:

Jakub Kubisz

Studijní program:

B0613A140014 Informatika

Téma:

Nástroj pro hashtag integraci v MS Outlook
Hashtag Plugin for MS Outlook

Jazyk vypracování:

čeština

Zásady pro vypracování:

Cílem práce je vytvořit nástroj pro integraci využití "hashtag" v Microsoft Outlook. Nástroj bude podporovat vyhledávání hashtagů ve všech artefaktech, zobrazí, kde jsou použity a možnost filtrování a náhledů na jednotlivé položky jako jsou úkoly, záznam v kalendáři apod.

1. Seznamte se s možností vytvoření pluginu pro MS Outlook.
2. Seznamte se s případnými stávajícími nástroji pro využití hashtagů.
3. Navrhněte a implementujte nástroj integrovaný jako plugin v MS Outlook.
4. Vyzkoušejte funkčnost nástroje na ukázkových příkladech.

Seznam doporučené odborné literatury:

[1] Pfleeger, Shari Lawrence, and Joanne M. Atlee. 2009. Software Engineering: Theory and Practice: Prentice Hall, ISBN 0136061699

[2] Pressman, Roger S. 2010. Software Engineering : A Practitioner's Approach. 7th ed. New York: McGraw-Hill Higher Education, ISBN 9780073375977

[3] Sommerville, Ian. 2010. Software Engineering. 9th ed, International Computer Science Series. Harlow: Addison-Wesley, ISBN 978-0137035151

Další literatura podle pokynů vedoucího bakalářské práce.

Formální náležitosti a rozsah bakalářské práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí bakalářské práce: **Ing. Svatopluk Štolfa, Ph.D.**

Datum zadání: 01.09.2023

Datum odevzdání: 30.04.2024

Garant studijního programu: doc. Mgr. Miloš Kudělka, Ph.D.

V IS EDISON zadáno: 09.11.2023 14:57:06

Abstrakt

Práce se zaměřuje na tvorbu doplňku pro MS Outlook určenému ke práci s hashtagy v rámci programu MS Outlook. Úvodem práce je motivace k použití hashtagů v MS Outlook a tvorbě doplňku, včetně vize tohoto systému. Práce dále rozebírá problematiku tvorby doplňků pro MS Outlook, typy doplňků, jejich možnosti a omezení a technologie, které doplňky využívají. Také obsahuje popis Objektového modelu Office pro Outlook, který je základní technologií pro tvorbu těchto doplňků. Dále práce obsahuje samotný návrh systému, určení jeho požadavků, výběr vhodného typu doplňků, technologii a návrh vhodné architektury, včetně odůvodnění pro tyto rozhodnutí. Další částí projektu je implementace doplňku, která se zaměřuje na výslednou implementaci projektu, uvádí zajímavé příklady řešení problémů pomocí návrhových vzorů a dostupných technologií a popisuje využití algoritmy a problémy při vývoji. Výsledkem práce je doplněk umožňující integraci hashtagů do aplikace MS Outlook, jak uvádí závěr práce. V rámci práce byl také vytvořen instalační a uživatelský manuál k doplňku.

Klíčová slova

Hashtag; MS Outlook; VSTO; Microsoft; .NetFramework; Integrace; Add-in; COM; C#; Office;

Abstract

The thesis focuses on the creation of an add-in for MS Outlook designed to manage hashtags within MS Outlook. The thesis starts with the motivation to use hashtags in MS Outlook and the creation of the add-in, including the vision of this system. The thesis then discusses the topic of creating add-ins for MS Outlook, the types of add-ins, their capabilities and limitations, and the technologies that add-ins use. It also includes a description of the Office Object Model for Outlook, which is the underlying technology for creating these add-ins. In addition, the thesis includes the actual process of designing the system, determining its requirements, selecting the appropriate type of add-ins, and designing architecture, including the justification for these decisions. The next part of the project is the implementation of the add-in, which focuses on the final implementation of the project, gives interesting examples of problem solutions using the design patterns and available technologies, and describes the algorithms and challenges encountered during development. The result of the work is an add-in that allows the integration of hashtags into MS Outlook, as the conclusion of the thesis states. The thesis also included an installation and user manual for the add-on.

Keywords

Hashtag; MS Outlook; VSTO; .NetFramework; Integration; Add-in; COM; C#; Office; Application

Obsah

Seznam použitých symbolů a zkratk	6
Seznam obrázků	7
Seznam tabulek	8
1 Úvod	9
1.1 Motivace	9
1.2 Vize aplikace	10
2 Průzkum stávajících nástrojů a technologií doplňků pro Outlook	11
2.1 Stávající nástroje	11
2.2 Typy doplňků pro Outlook	12
2.2.1 Visual Studio Tools for Office (VSTO)	12
2.2.2 Web Add-ins	15
2.3 Technologie doplňků pro Outlook	16
2.3.1 .NET Framework	16
2.4 Objektový model Office pro Outlook	16
2.4.1 Application objekt	16
2.4.2 Artefakty	18
3 Návrh systému	20
3.1 Požadavky	20
3.2 Volba typu doplňku	27
3.3 Architektura	28
3.3.1 Rozdělení do projektů	29
3.4 Optimalizace systému	31
3.4.1 Dependency Injection (DI)	31
3.4.2 Separation of Concerns	31
3.4.3 Model-View-ViewModel (MVVM)	31

3.4.4	Event Aggregator	32
3.4.5	Database Interaction	32
3.4.6	Factory Pattern	34
4	Implementace	35
4.1	Využité technologie	35
4.1.1	Databáze	35
4.1.2	IoC	36
4.1.3	Uživatelské rozhraní	37
4.2	Implementace technologií	38
4.2.1	Interakce s Objektovým Modelem Outlook	38
4.3	Využité algoritmy	42
4.3.1	Levenshteinova vzdálenost	42
4.4	Aplikace návrhových vzorů	48
4.4.1	Factory Pattern	49
4.5	Testování	50
4.6	Výzvy a problémy	51
5	Poznatky a závěr	53
5.1	Poznatky	53
5.2	Závěr	53
	Literatura	54
	Přílohy	55
A	Manuál	56
A.1	Instalační manuál	56
A.2	Uživatelský manuál	57
A.2.1	Vyhledávání	58
A.2.2	Přehled hashtagů	60
A.2.3	Detail Hashtagu	62

Seznam použitých zkratk a symbolů

MS	– Microsoft
VSTO	– Visual Studio Tools for Office
IoC	– Inversion of control
DI	– Dependency injection
UI	– User interface
MAPI	– Messaging Application Programming Interface
DI	– Dependency Injection
IoC	– Inversion of controll
MVVM	– Model-View-ViewMode

Seznam obrázků

2.1	Web add-in architektura [3]	13
3.1	UC1 - Vyhledávání	21
3.2	UC2 - Otevření detailu artefaktu	22
3.3	UC3 - Indexování	23
3.4	Projektový diagram	29
4.1	Levenshteinova vzdálenost [13]	42
4.2	Ukázka Levenshteinovy vzdálenosti [16]	43
A.1	Outlook s doplňkem hashtrack	57
A.2	záložka Vyhledávání	59
A.3	Přehled zaindexovaných hashtagů	61
A.4	Detailní pohled	62
A.5	Přidání nového hashtagu	64
A.6	Nastavení hashtagů	65

Seznam tabulek

3.1	Use Case Vyhledávání	21
3.2	Use Case Otevření detailu artefaktu	22
3.3	Use Case: Indexování	23
3.4	Use Case: Indexování artefaktů	24
3.5	Use Case: Spojování hashtagů	25
3.6	Use Case: Rozpojování hashtagů	26

Kapitola 1

Úvod

1.1 Motivace

Mnoho uživatelů e-mailových služeb čelí stále narůstajícímu objemu e-mailové komunikace, což s sebou přináší rostoucí náročnost na organizaci a udržení přehlednosti e-mailové schránky. Tento problém je obzvláště výrazný v korporátních prostředích, kde velké množství e-mailů a různorodost témat, od projektů přes zákazníky až po produkty, vyžaduje efektivní řešení pro správu a třídění informací.

Jedním z možných řešení je použití hashtagů, které nabízejí dynamický a flexibilní způsob organizace e-mailů. Na rozdíl od tradičních kategorií, které vyžadují manuální vytvoření, přiřazení nebo případně vytváření specifických pravidel pro každou kategorii, hashtagy umožňují dynamické rozdělení, jelikož nepotřebují být předem definované, díky tomu mohou být atomičtější a můžeme jich mít větší množství.

Uživatelé nemusí hashtagy vytvářet ručně a mohou si je za běhu přizpůsobovat, což je zvláště užitečné v prostředích, kde se témata nebo projekty rychle mění. S hashtagy je rozdělení automatické, bez nutnosti manuálně vytvářet nové kategorie pro každý projekt. Navíc, když uživatelé obdrží e-mail s novým hashtagem, může být okamžitě používán bez jakéhokoliv manuálního zásahu.

Motivací k průzkumu použití hashtagů jsou možnosti zvýšení efektivity, zjednodušení organizace a spolupráce.

1.2 Vize aplikace

Cílem práce je vytvořit doplněk, neboli plugin pro Microsoft Outlook, který umožní uživatelům efektivně spravovat a manipulovat s hashtagy v e-mailech a dalších artefaktech. Hlavním úkolem tohoto pluginu je zlepšení organizace e-mailové schránky tím, že poskytne nástroje pro práci s hashtagy. Tyto nástroje zahrnují možnosti jako jsou shlukování, vyhledávání a přehledy hashtagů.

Plugin by měl poskytnout uživatelům intuitivní a přehledné rozhraní pro organizaci jejich e-mailů podle hashtagů, které jsou pro ně důležitá. Díky této funkcionalitě budou moci uživatelé rychle filtrovat, vyhledávat a řadit e-maily podle specifických hashtagů, což jim umožní udržet si lepší přehled o svých projektech, komunikaci s klienty a interních procesech.

Klíčové funkce, které plugin nabídne, jsou:

Shlukování Hashtagů: Automatické seskupování hashtagů podle přidružených hashtagů, což umožní snadnější navigaci a správu.

Kategorizování Hashtagů: Automatické vytváření a přiřazování kategorií artefaktům s vybranými hashtagy.

Naše vize je, aby tento plugin nejen zjednodušil každodenní práci s e-maily a dalšími Outlook artefakty, ale také přinesl nový způsob, jakým lidé přistupují k organizaci a prioritizaci své komunikace. S těmito funkcemi nástroj cílí na každého uživatele, který hledá lepší způsob, jak zůstat organizovaný ve světě přetíženém informacemi.

Kapitola 2

Průzkum stávajících nástrojů a technologií doplňků pro Outlook

Tato kapitola zkoumá existující nástroje, pro práci a správu hashtagů v MS Outlook, poskytuje přehled o technologiích, které jsou základem pro vývoj pluginu pro MS Outlook. Zahrnuje diskusi o významu .NET Frameworku a technologiích VSTO, jejich role ve vývoji aplikací a porovnání s technologiemi jako jsou Web Add-ins. A také odůvodnění pro zvolení technologií.

2.1 Stávající nástroje

V rámci projektu byl proveden průzkum existujících nástrojů pro práci s hashtagy v aplikaci Outlook. Byl prozkoumán Microsoft AppSource [1], oficiální marketplace pro doplňky a aplikace integrovatelné se všemi produkty Microsoft Office. Cílem bylo identifikovat, zda již existují nástroje umožňující efektivní využití hashtagů v aplikacích MS Outlook.

Výsledky průzkumu ukázaly, že ačkoliv marketplace obsahuje řadu doplňků na práci s hashtagy pro aplikace jako SharePoint, Dynamics 365 Sales, či Power BI, řešení umožňující práci s hashtagy pro v MS Outlook nebylo nalezeno.

Byla prozkoumána i možnost využití vyhledávacího panelu, včetně pokročilého vyhledávání v MS Outlook. Tento průzkum však odhalil, že stávající vyhledávací funkce v Outlook nedokáží efektivně rozlišovat řetězce ve spojení s hashtagy. Jelikož vyhledávací nástroj ignoruje znak "#"v hledaném řetězci a hledá jen na základě podobnosti zbylého textu. Toto omezení představuje důvod pro vývoj nového nástroje.

2.2 Typy doplňků pro Outlook

Představíme si dva základní typy doplňků pro Outlook, Visual Studio Tools for Office (VSTO) a jejich alternativu Web Add-ins. Volba mezi těmito typy doplňků je klíčová jelikož, každý typ využívá jiné technologie a tak nám určuje tech-stack aplikace, čili jaké technologie se budou pro vývoj používat.

V této sekci si představíme technologie, které byly zvoleny k vývoji projektu, jejich alternativy a odůvodníme rozhodnutí pro tyto technologie. Představíme si .NET Framework, Visual Studio Tools for Office (VSTO) a jejich alternativu Web Add-ins.

2.2.1 Visual Studio Tools for Office (VSTO)

Visual Studio Tools for Office představuje soubor nástrojů, který umožňuje vývojářům vytvářet přizpůsobené řešení, která fungují přímo v produktech Microsoft Office. Tato technologie umožňuje rozšíření standardní funkcionality Office a její integrace s vlastními aplikacemi. VSTO je postaven na .NET Frameworku a umožňuje využívat jazyky C# nebo VB.NET pro tvorbu pluginů pro aplikace jako je Outlook, Excel, Word a další.

VSTO umožňuje vývojářům těsnou integraci .NET Framework aplikací s Office aplikacemi, vytváření uživatelských rozhraní a rozšíření uživatelských rozhraní Office Aplikací. Navíc VSTO poskytuje přístup k celému objektovému modelu Office, což dává vývojářům kontrolu nad aplikací a umožňuje jim vytvářet sofistikované a výkonné pluginy, které mohou rozšiřovat funkcionality Office aplikací.

Shrnutí vlastností VSTO Add-ins:

- Kompatibilita pouze s Windows.
- Vytvářeny s použitím .NET (C#, VB.NET).
- Umožňuje hlubokou integraci s Office aplikacemi.
- Rozsáhlé možnosti přizpůsobení UI, včetně vlastních task panes a form regions.
- Přímý přístup k Office objektovému modelu.
- Vyžaduje instalaci na každém klientském počítači.

2.2.1.1 Fungování VSTO doplňku s Microsoft Office

Doplňky VSTO vytvořené pomocí vývojářských nástrojů Office v sadě Visual Studio mají funkce architektury, které zvýrazňují stabilitu a zabezpečení a umožňují jim úzce spolupracovat s systémem Microsoft Office.

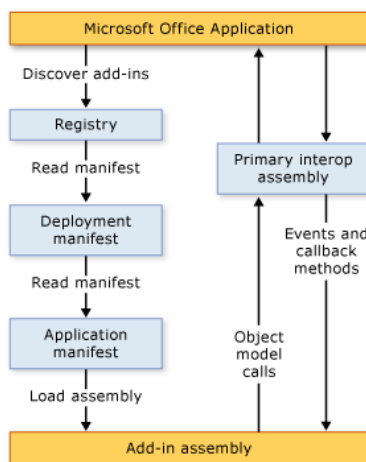
Po načtení sestavení může doplněk VSTO reagovat na události, které jsou vyvolány v aplikaci (například když uživatel klikne na položku nabídky). Doplněk VSTO může také volat do objektového modelu pro automatizaci a rozšíření aplikace a může používat libovolnou z tříd v rozhraní .NET Framework.

Pokud je pro aplikaci nainstalovaných více doplňků VSTO, každý doplněk VSTO se načte v jiné doméně aplikace. To znamená, že jeden doplněk VSTO, který se chová nesprávně, nemůže způsobit selhání jiných doplňků VSTO. Pomáhá také zajistit, aby se při zavření aplikace zrušila všechna sestavení doplňku VSTO z paměti.

Text byl citován z [2]

2.2.1.1.1 Položky registru jsou zásadní pro to, aby aplikace Microsoft Office mohly doplňky VSTO identifikovat a načítat. Visual Studio automaticky vytváří nezbytné položky registru během vývoje pro ladění a testování. Při nasazení doplňku se klíče registru vytvářejí na uživatelských počítačích pomocí ClickOnce nebo jiných nasazovacích technologií.

2.2.1.1.2 Manifest nasazení a manifest aplikace jsou dokumenty XML, které hrají klíčovou roli při aktualizacích a nasazení doplňků. Manifest nasazení udržuje odkazy na nejnovější manifesty aplikací, jež pak vedou k aktuálním sestavením doplňku VSTO a specifikují hlavní třídu, která se má spustit. *Text byl parafrázován z [2]*



Obrázek 2.1: Web add-in architektura [3]

1. „Aplikace zkontroluje položky registru, které identifikují doplňky VSTO vytvořené pomocí vývojářských nástrojů Office v sadě Visual Studio.“
2. „Pokud aplikace najde tyto položky registru, aplikace načte VSTOEE.dll, která načte VSTO-Loader.dll...“
3. „VSTOLoader.dll načte rozhraní .NET Framework a spustí spravovanou část sady Visual Studio Tools for Office runtime .“
4. „Visual Studio Tools for Office runtime kontroluje aktualizace manifestu a stáhne nejnovější manifesty aplikací a nasazení.“
5. „Visual Studio Tools for Office runtime provádí řadu kontrol zabezpečení.“
6. „Pokud je doplněk VSTO důvěryhodný ke spuštění, visual Studio Tools for Office runtime používá manifest nasazení a manifest aplikace ke kontrole aktualizací sestavení. Pokud je k dispozici nová verze sestavení, modul runtime stáhne novou verzi sestavení do mezipaměti ClickOnce v klientském počítači.“
7. „ Visual Studio Tools for Office runtime vytvoří novou doménu aplikace, do které se má načíst sestavení doplňku VSTO. “
8. „ Visual Studio Tools for Office runtime načte sestavení doplňku VSTO do domény aplikace. “
9. „ Visual Studio Tools for Office runtime volá metodu RequestComAddInAutomationService v doplňku VSTO, pokud jste ji přepsali. Volitelně můžete tuto metodu přepsat tak, aby se objekt v doplňku VSTO zpřístupnil jiným Microsoft Office řešením. “
10. „Visual Studio Tools for Office runtime volá metodu RequestService v doplňku VSTO, pokud jste ji přepsli. “
11. „ Visual Studio Tools for Office runtime volá metodu ThisAddIn_Startup v doplňku VSTO. Tato metoda je výchozí obslužnou rutinou Startup události události.“

Poznámka: Pro detailní popis kroků v procesu načítání doplňků VSTO, viz oficiální dokumentace Microsoftu. [2]

2.2.2 Web Add-ins

Web Add-ins neboli také Office Add-ins, dříve známé jako Office Web Apps a Apps for Office, jsou moderním řešením pro rozšiřování funkcí aplikací Microsoft Office nezávisle na platformě. Díky použití standardních webových technologií umožňují vývojářům vytvářet přizpůsobení, která mohou být snadno přenesena mezi různými zařízeními a platformami, včetně Windows, Mac, iOS a Android, stejně jako Office na webu.

Office Add-ins fungují skrze centrální model, kde jsou nasazeny a spravovány prostřednictvím Microsoft 365 admin center nebo přímo z Office Store. To poskytuje administrátorům možnost spravovat přístup a distribuci add-inů napříč organizací, což je v kontrastu s potřebou lokální instalace VSTO Add-ins na jednotlivých uživatelských počítačích.

Shrnutí vlastností Web Add-ins:

- Multi-platformní kompatibilita (Web, Windows, Mac, iOS, Android).
- Vyvíjeny s použitím HTML, CSS, JavaScript.
- Centralizované nasazení přes Microsoft 365 admin center nebo Office Store.
- Omezené možnosti, kvůli Office JavaScript API.
- Podpora UI elementů jako jsou task panes, dialogová okna, kontextové panely.

Office Add-ins využívají Office.js API, pro komunikaci s objektovým modelem Office tímto mohou být omezeny v přístupu k některým funkcím, které VSTO Add-ins mohou nabídnout, zejména v oblastech pokročilé interakce s aplikacemi Office a přizpůsobení uživatelského rozhraní. To z Web Add-ins činí vhodnou volbu pro řešení, která vyžadují širokou dostupnost a jsou méně závislá na hluboké integraci s Office aplikacemi.

2.3 Technologie doplňků pro Outlook

2.3.1 .NET Framework

.NET Framework je softwarový framework vyvinutý společností Microsoft, který poskytuje podporu pro vývoj a spuštění aplikací na Windows platformě. Jde o komplexní platformu s bohatým API pro vývoj desktopových, webových a mobilních aplikací. Využívá se k vývoji VSTO Outlook doplňků.

.NET Framework v4.8 je nejnovější a také poslední verzí, jde také o poslední verzi, kterou VSTO doplňky podporují, jelikož VSTO využívá COM technologií pro interakci s Objektovým modelem Outlook, spoléhající se na .NET Framework, tato technologie již není nepodporovaná v .NET 5+ ani .NET Core.

[4]

2.4 Objektový model Office pro Outlook

V této sekci si představíme Objektový model Office pro Outlook a jeho objekty nejvyšší urovně. Objektový model Office pro Outlook poskytuje vývojářům rozhraní, prostřednictvím kterého mohou interagovat s různými komponentami aplikace Outlook. Základním objektem tohoto modelu je **Application objekt**.

„Objektový model Aplikace Outlook poskytuje třídy a rozhraní, které představují položky v uživatelském rozhraní. Například objekt Application představuje celou aplikaci, Folder objekt představuje složku, která obsahuje e-mailové zprávy nebo jiné položky a MailItem objekt představuje e-mailovou zprávu.“ [5]

*Poznámka: V této sekci se zabýváme objektovým modelem Office pro **Outlook**, modely pro jiné aplikace se mohou lišit.*

2.4.1 Application objekt

Objekt Application je vstupním bodem do objektového modelu Outlooku a slouží jako centrální místo pro přístup k ostatním objektům a funkcím v aplikaci. Tento objekt umožňuje vývojářům interakci s aplikací Outlook, jako je správa e-mailů, kalendářů, kontaktů a úkolů, reagování na události vyvolané v aplikaci Outlook, nebo implementace pokročilých funkcí, jako jsou automatizace a přizpůsobení uživatelského rozhraní.

2.4.1.1 Funkcionalita Application objektu

Mezi klíčové funkce objektu *Application* patří:

- **Správa položek:** Objekt umožňuje vytvářet, číst, upravovat a mazat e-mailové zprávy, kalendářní události, kontakty a úkoly.

- **Interakce s uživatelským rozhraním:** Vývojáři mohou pomocí objektu *Application* manipulovat a přizpůsobovat prvky uživatelského rozhraní, jako jsou formuláře, panely nástrojů a menu.
- **Zpracování událostí:** Objekt poskytuje přístup k událostem, jako jsou příchozí zprávy, změny v kalendáři nebo změny stavu úkolů, což umožňuje reagovat dynamicky na uživatelské interakce nebo systémové události.

Informace k této sekci byly čerpány z oficiální dokumentaci Microsoftu [6]

2.4.1.2 Události

Application objekt také umožňuje odchyťovat a reagovat na *Události*, neboli *Events*, vyvolané aplikací, mezi nejzákladnější události na které **Application** objekt umožňuje reagovat patří:

- **AdvancedSearchComplete:** Tato událost je vyvolána po dokončení pokročilého vyhledávání, které bylo zahájeno pomocí metody 'AdvancedSearch' objektu 'Application'. Poskytuje zpětnou vazbu, když je vyhledávání dokončeno, což umožňuje převzít a zpracovat výsledky vyhledávání.
- **MAPILogonComplete:** Tato událost je vyvolána poté, co je uživatel úspěšně přihlášen k poštovnímu serveru přes MAPI (Messaging Application Programming Interface). Je užitečná pro inicializaci funkcí, které závisí na plném přístupu uživatele k jeho e-mailovým zprávám a ostatním poštovním objektům.
- **NewMail:** Událost je vyvolána vždy, když do poštovní schránky přijde nová e-mailová zpráva. Tato událost umožňuje okamžitě reagovat na příchozí poštu, například pro automatické zpracování zpráv.
- **Reminder:** Událost je vyvolána, když je aktivováno upozornění (*vyskakovací okno* pro jakýkoliv kalendářní objekt, úkol nebo e-mailovou zprávu).
- **Startup:** Událost se vyvolá při spuštění aplikace Outlook. Tato událost je využívána pro inicializaci doplňků nebo jiných komponent, které musí být nastaveny hned po spuštění aplikace.
- **Quit:** Událost je spuštěna při zavírání aplikace Outlook. Tato událost umožňuje korektně ukončit doplňky, je ideální pro uvolnění zdrojů nebo ukládání dat a nastavení do perzistentního uložště.

Informace k této sekci byly čerpány z oficiální dokumentaci Microsoftu [6]

2.4.2 Artefakty

V kontextu tohoto projektu termín **artefakty** označuje základní objekty v objektovém modelu Office pro Outlook. Tyto artefakty zahrnují objekty, jako jsou e-maily, kontakty, úkoly a kalendářní položky, a jsou hlavními objekty se kterými se v MS Outlook pracuje a se kterými uživatel interaguje. Tyto objekty budou tedy základem pro interakci s uživatelskými daty.

Každý z artefaktů má specifické vlastnosti a metody, které umožňují provádět operace, jako je čtení, úprava, mazání a sledování změn. Artefakty jsou zásadní pro realizaci funkčnosti, která reaguje na uživatelské akce a přistupuje k datům uživatele.

Mnoho objektů v objektovém modelu sdílí řadu společných vlastností, které umožňují manipulovat s těmito objekty, reagovat na události a integrovat funkce do svých aplikací. Ačkoliv Artefakty nemají společný interface sdílí několik společných vlastností:

- **EntryID:** Každý objekt v Outlooku, jako je e-mail, kontakt, úkol nebo událost, má unikátní identifikátor známý jako *EntryID*. Tento identifikátor umožňuje programové načtení a reference k specifickým instancím objektů. Pro práci s ním je však důležité si uvědomit, že *EntryID* se může změnit při určitých operacích, jako je například přesunutí e-mailu do jiné složky. Kvůli této vlastnosti *EntryID* není vhodným identifikátorem k trvalé referenci na konkrétní artefakt.
- **Inspector objekt:** Pro zobrazení nebo editaci objektu v samostatném okně můžete použít *Inspector* objekt. Tento objekt umožňuje otevření nového okna pro každý objekt, což je užitečné pro zobrazení detailu artefaktu a nebo jeho úpravy.
- **Vlastnosti a metody:** Objekty mají standardní sadu vlastností a metod, které umožňují zobrazení a úpravu jejich atributů, jako jsou předmět e-mailu, datum schůzky, jméno kontaktu atd. Metody také umožňují provádění běžných akcí, jako je uložení změn, zavření objektu nebo jeho smazání.
- **Události:** Každý Artefakt vyvolává události, jako je změna, otevření, zavření, uložení nebo smazání. Tato funkčnost umožňuje aplikacím reagovat na akce uživatele nebo systémové události.

2.4.2.1 MailItem

Objekt *MailItem* je představuje e-mailovou zprávu, umožňuje přístup k datům. správu, editaci a odchyťávání událostí na konkrétní e-mailové zprávě.

MailItem tak jako jiné artefakty umožňuje provádět na zprávě různé akce jako je kopírování, přesun zprávy do jiné složky, smazání zprávy, nebo zobrazení zprávy v novém *Inspector* objektu (nové okno). Také umožňuje reagovat na eventy, které aplikace vyvolává, jako je uložení rozepsané zprávy, přečtení, smazání, zavření, odeslání nebo i odeslání odpovědi na zprávu a další viz. [7]

Informace k této sekci byly čerpány z oficiální dokumentaci Microsoftu [7]

2.4.2.2 AppointmentItem

AppointmentItem reprezentuje kalendářní událost v Outlooku. Tento objekt umožňuje vytváření, editaci a správu schůzek a událostí, včetně nastavení účastníků, připomínek a opakování události. *AppointmentItem* také podporuje odchyťávání událostí jako jsou změny, zrušení, nebo odeslání pozvánky. [8]

2.4.2.3 ContactItem

Objekt *ContactItem* zastupuje kontakt v Outlooku. Umožňuje uživatelům ukládat a spravovat informace o kontaktech, jako jsou jména, telefonní čísla, adresy a další osobní údaje. Kontakt může být také spojen s různými událostmi, jako jsou e-maily nebo schůzky, což usnadňuje organizaci komunikace. [9]

2.4.2.4 TaskItem

TaskItem je objekt pro správu úkolů v Outlooku. Umožňuje uživatelům vytvářet, editovat a sledovat úkoly, nastavovat jejich prioritu, stav a termíny. Úkoly mohou být také propojeny s připomínkami a aktualizace stavu mohou být sdíleny s ostatními uživateli nebo systémy.[10]

Informace k této sekci byly čerpány z oficiální dokumentaci Microsoftu [7, 9, 10, 8]

Kapitola 3

Návrh systému

Tato kapitola se detailněji zaměřuje na požadavky systému a výběr vhodné architektury systému. Při návrhu byl kladen důraz na modularitu, snadnou údržbu a rozšiřitelnost. Rozdělení systému do více projektů umožňuje dosáhnout lepší separace zodpovědností a zjednodušení testování. Tento přístup také usnadňuje implementaci nových funkcionalit a jejich integraci do stávajícího systému bez potřeby zásadních změn v již existujících komponentách.

3.1 Požadavky

V této sekci, jsou popsány požadavky systému za použití případů užití (use-case).

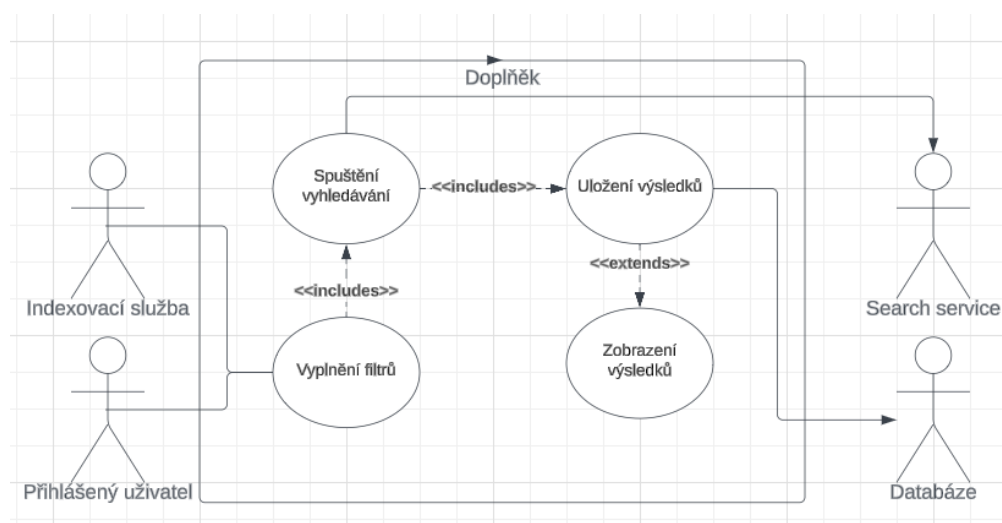
Vyhledávání

Vyhledávání artefaktů probíhá pro schránku uživatele, na základě specifikovaných pravidel, může probíhat pro specifický hashtag, několik hashtagů nebo pro všechny výskyty hashtagů. Nejprve se určí rozsah, poté se provede prohledání všech artefaktů ve schránce pro výskyt specifického nebo libovolného hashtagu. Výsledky jsou následně **UC1: Vyhledávání**

Uživatel systému vyhledává specifický hashtag ve zvolených artefaktech (např. e-mailech) v rámci své poštovní schránky. Nalezené artefakty jsou zpracovány do zobrazitelné formy, uloženy do cache a databáze. Výsledky jsou poté zobrazeny uživateli.

Tabulka 3.1: Use Case Vyhledávání

Element	Popis
ID	UC1
Název případu užití	Vyhledávání
Popis	Umožňuje uživatelům vyhledat specifický hashtag a napojené hashtagy v artefaktech jako jsou e-maily a tyto artefakty zobrazit.
Primární aktéři	Uživatel systému
Sekundární aktéři	Indexovací služba
Předpoklady	Uživatel musí být přihlášen, musí být zvolen minimálně jeden typ artefaktu, a musí být vyplněn vyhledávací text.
Hlavní kroky	1. Vyplnění filtrů vyhledávání. 2. Spuštění vyhledávání. 3. Systém uloží výsledky vyhledávání. 4. Systém zobrazí výsledky uživateli.
Alternativní kroky	Vyhledávání ve všech výsledcích nebo jen v za-indexovaných (plánovaná možnost).
Postkondice	Výsledky vyhledávání jsou uživateli zobrazeny a jsou uloženy v systému.
Výjimky	Uživatel nevyplní všechny povinné filtry: Systém upozorní na chybu.



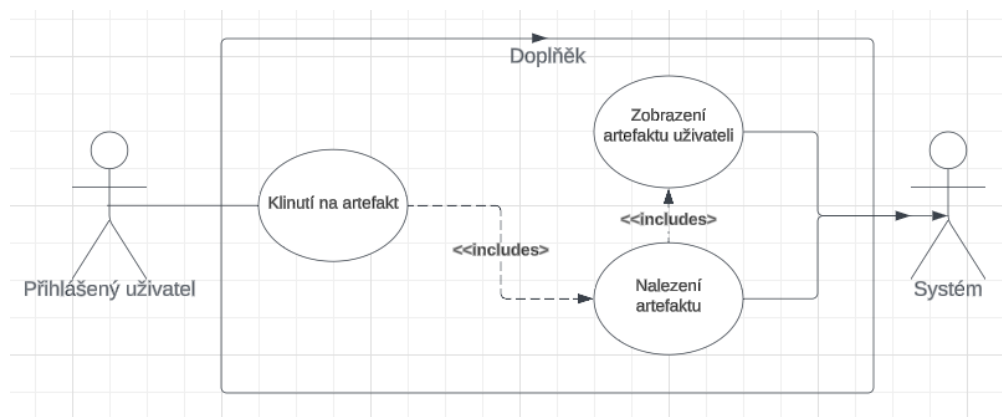
Obrázek 3.1: UC1 - Vyhledávání

UC2: Otevření detailu artefaktu

Uživatel systému otevře detail artefaktu kliknutím na artefakt ve výsledku vyhledávání, detail artefaktu se otevře v novém okně.

Tabulka 3.2: Use Case Otevření detailu artefaktu

Element	Popis
ID	UC2
Název případu užití	Otevření detailu artefaktu
Popis	Umožňuje uživatelům zobrazit všechny detaily vybraného artefaktu v novém okně.
Primární aktéři	Uživatel systému
Předpoklady	Uživatel musí být přihlášen, musí být dokončeno vyhledávání, které našlo alespoň jeden výsledek.
Hlavní kroky	1. Uživatel ve výsledcích vyhledávání klikne na konkrétní artefakt. 2. Systém nalezne vybraný artefakt. 3. Systém zobrazí uživateli nativní detail artefaktu v novém okně.
Postkondice	Artefakt je uživateli zobrazen.



Obrázek 3.2: UC2 - Otevření detailu artefaktu

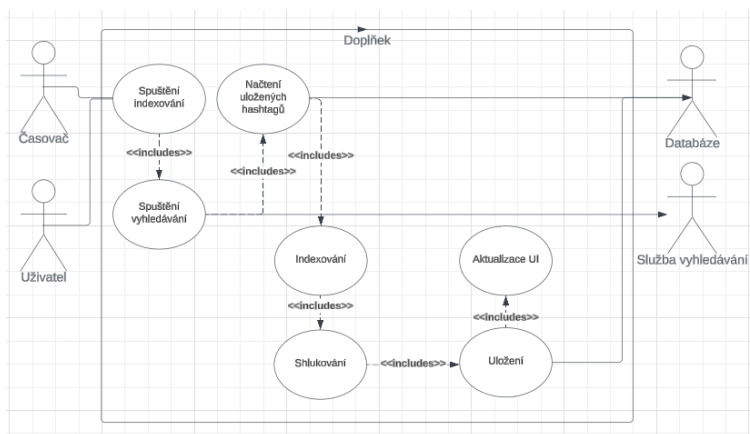
UC3: Indexování

Vzhledem ke složitosti use-case indexování byl vytvořen sub-use-case "UC4: Indexování artefaktů", který jsou popsán níže.

Indexování artefaktů probíhá pro schránku uživatele, na základě automatického časovaného spuštění systémem nebo manuálním spuštěním uživatelem. Indexování začíná Vyhledáním artefaktů k za-indexování, následuje načtení známých hashtagů a jejich dat z databáze, poté provádí Indexing artefaktu, jehož součástí je přiřazení artefaktů k hashtagům, poté se provádí slučování hashtagů a nakonec uložení do databáze a cache a informování UI o změnách.

Tabulka 3.3: Use Case: Indexování

Element	Popis
ID	UC3
Název případu užití	Indexování
Popis	Indexování vyhledá artefakty pro za-indexování, za-indexuje artefakty, zpracuje nalezené hashtagy a uloží výsledky.
Primární aktéři	Indexovací služba, Časovač
Sekundární aktéři	Databáze, Služba vyhledávání
Předpoklady	Uživatel je přihlášen.
Hlavní kroky	1. Spuštění indexování 2. Spuštění vyhledávání artefaktů. 3. Načtení hashtagů uložených hashtagů. 4. Indexování artefaktů. 5. Shlukování hashtagů. 6. Uložení výsledků. 7. Informování UI o aktualizaci
Alternativní kroky	Manuální spuštění indexování uživatelem.
Postkondice	Artefakty a hashtagy jsou indexovány a uloženy.
Výjimky	Chyby v načítání dat, chyby v indexování.



Obrázek 3.3: UC3 - Indexování

UC4: Indexování artefaktů Indexování artefaktů zahrnuje několik kroků. Nejprve se z databáze načtou již indexované hashtagy a příslušné artefakty. Následuje extrakce hashtagů z artefaktu a obohacení artefaktu o "custom properties", což jsou seznamy hashtagů vyextrahovaných z artefaktu. Dále se načte nebo vygeneruje unikátní ID pro artefakt, v závislosti na jeho existenci. Pokud vyextrahovaný hashtag není nalezen v seznamu načtených z databáze, vytvoří se nový hashtag s výchozím nastavením a přiřadí se k artefaktu. Nakonec se artefakt přiřadí do kategorie dle nastavení hashtagu. Výstupem procesu je seznam hashtagů nalezených v artefaktech, kde každý hashtag má přiřazené artefakty, ve kterých byl identifikován.

Tabulka 3.4: Use Case: Indexování artefaktů

Element	Popis
ID	UC5
Název případu užití	Indexování artefaktů
Popis	Proces indexování zahrnuje načítání existujících hashtagů a artefaktů, extrakci a přiřazení nových hashtagů k artefaktům, jejich obohacení o "custom properties" a kategorizaci. Výstupem je aktualizovaný seznam hashtagů obohacený o nově nalezené artefakty.
Primární aktéři	Indexovací služba
Sekundární aktéři	Databáze
Předpoklady	Je nalezen alespoň jeden artefakt.
Hlavní kroky	1. Načtení existujících hashtagů a artefaktů z databáze. 2. Extrakce hashtagů z nalezených artefaktů. 3. Obohacení artefaktů o "custom properties". 4. Generování nebo načtení unikátního ID pro artefakt. 5. Vytvoření nových hashtagů, pokud není nalezen existující pro hashtagy v artefaktu. 6. Kategorizace artefaktů dle nastavení hashtagu.
Alternativní kroky	-
Postkondice	Artefakty jsou správně indexovány a kategorizovány s aktualizovanými informacemi v databázi.
Výjimky	-

UC5: Spojování hashtagů

Spojování hashtagů, provádí uživatel v uživatelském rozhraní v nastavení konkrétního hashtagu, ve kterém uživatel vybírá jeden nebo více hashtagů ke sjednocení do hlavního hashtagu na němž toto sjednocení nastavuje. Tento proces začíná, když uživatel vstoupí do sekce nastavení hashtagu ve svém uživatelském rozhraní. Uživatel ze seznamu vybírá specifické hashtagy, které chce spojit. Po potvrzení spojení systém provede aktualizaci nastavení hashtagu, uloží změny do databáze, cache a nakonec informuje uživatelské rozhraní o aktualizaci.

Tabulka 3.5: Use Case: Spojování hashtagů

Element	Popis
ID	UC5
Název případu užití	Spojování hashtagů
Popis	Uživatel sjednocuje jeden nebo více hashtagů do hlavního hashtagu v rámci nastavení tohoto hashtagu v uživatelském rozhraní. Proces zahrnuje výběr hashtagů ke spojení, uložení nastavení spojení a aktualizaci uživatelského rozhraní.
Primární aktéři	Uživatel systému
Sekundární aktéři	Databáze, Cache, Uživatelské rozhraní (UI)
Předpoklady	Uživatel je přihlášen a má načtený alespoň 2 nespojené hashtagy.
Hlavní kroky	1. Uživatel vstoupí do sekce nastavení hashtagu. 2. Uživatel vybere hashtagy, které chce spojit. 3. Uživatel potvrdí spojení. 4. Systém aktualizuje nastavení hashtagu. 5. Systém uloží změny do databáze a cache. 6. Systém informuje UI o aktualizaci.
Alternativní kroky	-
Postkondice	Hashtagy jsou úspěšně spojeny, a data jsou aktualizována v UI a uložena v databázi a cache.
Výjimky	-

UC6: Rozpojování hashtagů

Rozpojování hashtagů, provádí uživatel v uživatelském rozhraní v nastavení konkrétního hashtagu, ve kterém uživatel vybírá jeden nebo více hashtagů ke rozpojení do hlavního hashtagu na němž toto rozpojení nastavuje. Tento proces začíná, když uživatel vstoupí do sekce nastavení hashtagu ve svém uživatelském rozhraní. Uživatel ze seznamu spojených hashtagů vybírá specifické hashtagy, které chce rozpojit. Po potvrzení rozpojení systém provede aktualizaci nastavení hashtagu, uloží změny do databáze, cache a nakonec informuje uživatelské rozhraní o aktualizaci.

Tabulka 3.6: Use Case: Rozpojování hashtagů

Element	Popis
ID	UC6
Název případu užití	Rozpojování hashtagů
Popis	Uživatel rozpojuje jeden nebo více hashtagů v rámci nastavení tohoto hashtagu v uživatelském rozhraní. Proces zahrnuje výběr hashtagů k rozpojení, uložení nastavení rozpojení a aktualizaci uživatelského rozhraní.
Primární aktéři	Uživatel systému
Sekundární aktéři	Databáze, Cache, Uživatelské rozhraní (UI)
Předpoklady	Uživatel je přihlášen a má alespoň jeden spojený hashtag.
Hlavní kroky	1. Uživatel vstoupí do sekce nastavení hashtagu. 2. Uživatel vybere ze seznamu spojených hashtagů, hashtagy které chce rozpojit. 3. Uživatel potvrdí rozpojení. 4. Systém aktualizuje nastavení hashtagu. 5. Systém uloží změny do databáze a cache. 6. Systém informuje UI o aktualizaci.
Alternativní kroky	-
Postkondice	Hashtagy jsou úspěšně rozpojeny, a data jsou aktualizována v UI a uložena v databázi a cache.
Výjimky	-

3.2 Volba typu doplňku

Výběr typu doplňku je klíčovým rozhodnutím, které nám určí tech-stack doplňku a významně ovlivňuje celý proces vývoje i nasazení doplňku. Pro tento projekt bylo rozhodnuto využít VSTO (Visual Studio Tools for Office) Add-ins z několika důvodů:

Převažující použití ve firmách: Ačkoliv Web Add-ins nabízí cross-platform kompatibilitu, většina organizací stále využívá Outlook na systémech Windows ("Outlook for Windows"), kde VSTO Add-ins jsou ideální volbou, díky možnosti hluboké integraci s Office aplikacemi. VSTO poskytuje rozsáhlé možnosti uživatelského přizpůsobení, což je zásadní pro firmy, které vyžadují pokročilé funkce a úpravy.

Výkon a bezpečnost: VSTO Add-ins poskytují přímý přístup k Office objektovému modelu a manipulaci s aplikacemi a daty bez významných omezení, což umožňuje hlubší a výkonnější integraci. Toto může být zvláště důležité pro aplikace, které vyžadují vysoký výkon a bezpečné zpracování citlivých dat.

Stabilita a zralost řešení: VSTO technologie, s její dlouhou historií vývoje a používání, nabízí stabilní a osvědčenou platformu pro vývoj. Tato vyzrálost znamená, že existuje široká škála dokumentace a podpůrných komunit. **Instalace a nasazení:** VSTO Add-ins mohou být jednoduše instalovány lokálně bez potřeby jakýchkoli serverových závislostí, na rozdíl od Web Add-ins, které vyžadují hosting na Microsoft 365 a povolení administrátora organizace pro instalaci, což může komplikovat jejich použití v organizacích.

Technické a operační omezení Web Add-ins: Web Add-ins jsou vyvíjeny s použitím HTML, CSS a JavaScriptu a jsou omezeny možnostmi Office JavaScript API. Tyto omezení mohou bránit implementaci některých pokročilých funkcí, které jsou realizovatelné pomocí VSTO, což má dopad na možné funkcionality.

Tato kombinace faktorů činí VSTO stále atraktivní volbou, přestože se trend pohybuje směrem k modernějším, webovým doplňkům. V kontextu tohoto projektu tedy volba VSTO Add-ins, je vhodná jelikož umožňuje využít stávající, časem ověřenou platformu a přímou integraci s Office objektovým modelem, což umožňuje více funkcionalit a vyšší výkonost.

3.3 Architektura

Tato sekce obsahuje návrh architektury projektu.

Na základě analýzy předchozích částí textu, zejména vize, volby typu doplňků a případů užití je zjevné, že bude za potřebí komplexní .Net Framework projekt, který bude pracovat s Objektovým Modelem Outlook, bude mít přístup do databáze, a bude mít interaktivní uživatelské rozhraní. V rámci správného designu a oddělení povinností, je tedy vhodné rozdělit doplněk na několik projektů, kde každý bude mít své povinnosti a nebudou navzájem moc provázány.

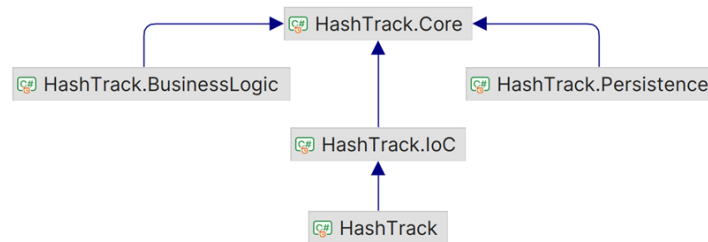
Vhodné rozdělení projektů podle povinností by bylo následující, projekt obstarávající všechny perzistentní povinnosti, projekt obstarávající uživatelské rozhraní a interakce s ním, nejlépe za pomoci MVVM návrhového vzoru, projekt obstarávající interakce s Objektovým Modelem Outlook, projekt s doménovou logikou, případně také Core projekt, pro rozhraní, enumerace, konstanty, rozšiřující metody a další části, které jsou využívány více projekty a v neposlední řadě spouštěcí projekt, který bude mít na starosti správnou inicializaci všech komponent a spuštění programu. Vzhledem ke komplexnosti doplňku bude nezbytné využití "Inversion of Control" a tedy by bylo vhodné mít projekt obstarávající tuto zodpovědnost.

Toto vhodné rozdělení ovšem nemusí být úplně ideální a může přidávat na komplexitě jednodušších programů, v případě tohoto doplňku není toto rozdělení nejvhodnější a tak v rámci tohoto doplňku bylo zvoleno jednodušší rozdělení zodpovědností. Spouštěcí projekt a uživatelské rozhraní bylo pro jednoduchost spojeno a interakce s Objektovým Modelem Outlook byla přiřazena jako povinnost projektu s doménovou logikou. Aplikace byla rozdělena do pěti hlavních projektů, přičemž každý z nich má specifické úkoly a odpovědnosti. Toto rozdělení napomáhá udržet kód čistý, organizovaný a snadno udržitelný, rozdělení a povinnosti každého projektu jsou popsány níže.

Poznámka: Informace k modularizaci aplikace v této části byly čerpány z [11]

3.3.1 Rozdělení do projektů

V této sekci je uveden přehled projektů a popis obsahu a povinností.



Obrázek 3.4: Projektový diagram

3.3.1.1 Projekt: HashTrack

Tento projekt je vstupní bod aplikace, má na starosti inicializaci a všechny aspekty uživatelského rozhraní a interakce s uživatelem. Zahrnuje implementaci uživatelského rozhraní a inicializaci, v rámci, které volá IoC projekt pro vytvoření DI kontejneru a následně do něj registruje instanci Objektového modelu Outlook Application, což je nezbytné pro práci s Objektovým Modelem Outlook mimo spouštěcí projekt.

3.3.1.2 Projekt: BusinessLogic

Projekt BusinessLogic obsahuje veškeré implementace doménové logiky potřebné pro zpracování dat, operace s hashtagy a jejich vyhledávání a hlavně zajišťuje hlavní interakce s Objektovým Modelem Outlook, což v tomto případě zapadá do zodpovědnosti domové logiky. Tento projekt je navržen tak, aby byl nezávislý na uživatelském rozhraní.

3.3.1.3 Projekt: Core

Core projekt slouží jako centrální místo pro sdílení kódu napříč projekty. Obsahuje definice rozhraní, sdílené doménové modely, konstanty, enumerace a rozšiřující metody, které jsou využívány více komponenty systému.

3.3.1.4 Projekt: IoC

Projekt IoC, jehož hlavní zodpovědností je "Inversion of control", řídí dependency injection a správu životního cyklu objektů. Tento projekt je klíčový pro správnou modularitu a snadnou konfiguraci závislostí mezi projekty. Využívá atributů na úrovni tříd k dynamickému nalezení a zaregistrování objektů do kontejneru.

3.3.1.5 Projekt: Persistence

Projekt Persistence se stará o všechny aspekty trvalého ukládání dat. Využívá Repository pattern k abstrakci přístupu k datům, což umožňuje snadnější změny v implementaci úložiště dat bez ovlivnění ostatních částí systému. Dále také obstarává uložení cache pro rychlejší odezvu systému.

3.4 Optimalizace systému

Tato sekce se zaměřuje na klíčové návrhové vzory a architektonické principy, které byly uvažovány při analýze a návrhu systému a byly zásadní pro optimalizaci a zlepšení návrhu systému.

Návrhové vzory mohou dramaticky zlepšit kvalitu softwarových systémů... Protože vzory poskytují osvědčená řešení pro běžné problémy, umožňují vývojářům opakovaně používat sofistikované techniky, aniž by museli znovu objevovat kolo. *Parafrázováno z [12]*

3.4.1 Dependency Injection (DI)

Dependency Injection je technika používaná k dosažení inverze závislostí mezi komponentami. DI usnadňuje testování a správu kódu díky redukci pevných závislostí.

Tento návrhový vzor je jedním z klíčových návrhových vzorů zvolených při návrhu doplňku, jelikož umožňuje snadnou práci s více projekty doplňku a záměnu implementací, čímž redukuje pevné závislosti a usnadňuje testování a modifikace a údržbu kódu. Tento vzor je využit na každé třídě se závislostmi na jiné službě.

3.4.1.1 Využití životní cykly

- **Singleton:** Tento životní cyklus zaručuje, že třída má pouze jednu instanci po celou dobu běhu aplikace.
- **Transitive:** Objekty jsou vytvářeny nové vždy, když jsou požadovány.

3.4.2 Separation of Concerns

Tento princip podporuje oddělení různých funkcionalit do samostatných částí projektu, což usnadňuje správu a rozvoj aplikace. Doplňěk byl rozdělen na několik projektů kde každý se stará o své povinnosti.

3.4.3 Model-View-ViewModel (MVVM)

Tento návrhový vzor, byl zvolen pro uživatelské rozhraní, udržení přehlednosti a atomičtější separace povinností. Obstarává veškeré interakce uživatele s aplikací a předává je aplikační vrstvě, má na starosti udržovat UI aktuální a reagovat na události informující o aktualizaci dat a načítání těchto dat.

3.4.3.1 Command Pattern

Obstarává transport akcí uživatele z View do ViewModelu a umožňuje na ně reagovat a zároveň dodržet zapouzdření.

3.4.4 Event Aggregator

Je klíčový vzor, který umožňuje separaci mezi projekty, díky možnosti přihlašovat se k událostem, spouštět události a reagovat na události bez nutnosti reference na komponentu jež událost vyvolala. Tento vzor je hojně využit, na propagaci událostí mezi komponenty, například je vhodný pro aktualizaci UI po dokončení zpracování dat a jejich aktualizaci.

3.4.5 Database Interaction

3.4.5.1 Repository Pattern

Repository Pattern odděluje logiku přístupu k datům od doménové logiky, což usnadňuje údržbu a testování komponent aplikace.

Vhodné pro implementační část

3.4.5.2 Attribute - Decorator Pattern

Decorator Pattern [12] umožňuje dynamicky přidávat nebo modifikovat chování objektů bez změny jejich kódu. Využívá se pro rozšíření funkcionalit objektů za běhu.

V projektu je využit pro dynamickou Registraci objektů do DI kontejneru (4.11) a určení jejich životního cyklu. Umožňuje nám registrovat služby přímo v jejich kódu, což zpřehledňuje kód a zjednodušuje údržbu. Při registraci volíme pud jakým typem bude služba v DI kontejneru zaregistrována a její životní cyklus.

Také se využívá pro registrování speciálních služeb (4.10) pro obsluhu událostí, tato registrace probíhá za pomoci textového řetězce identifikujícího událost a typ pro registraci do DI kontejneru, tyto služby se vždy registrují jako Transientní jelikož po zpracování události již nejsou potřeba a tak je žádoucí aby zanikly.

```
[RegisterService(LifeCycle.Transient, typeof(
    ISearchCompletedHandlerFactory))]
public class SearchCompletedHandlerFactory :
    ISearchCompletedHandlerFactory
```

Listing 3.1: Využití atributu pro Registrování služby

```
[RegisterHandler(Events.DefaultSearchCompleted, typeof(
    ISearchCompletedHandler))]
public class AdvancedSearchCompletedHandler : ISearchCompletedHandler
```

Listing 3.2: Využití atributu pro Registrování event handleru

3.4.6 Factory Pattern

Factory Pattern [12] se používá k vytváření objektů bez specifikace konkrétních tříd objektů. Factory metoda deleguje vytváření objektů.

Tento návrhový vzor je v projektu využit pro obsluhu událostí (4.12), služba zachytí obecnou událost a na základě jejích dat rozhodne, handler kterého typu má vytvořit.

```
public class SearchCompleteHandlerFactory :
    ISearchCompleteHandlerFactory
{
    private readonly IComponentContext _context;
    public SearchCompleteHandlerFactory(IComponentContext context)
    {
        _context = context;
    }

    public void HandleSearchCompleted(Outlook.Search SearchObject)
    {
        var handler = _context.ResolveKeyed<ISearchCompleteHandler>(
            SearchObject.Tag);
        handler.HandleSearchComplete(SearchObject);
    }
}
```

Listing 3.3: Využití návrhového vzoru Factory

Kapitola 4

Implementace

4.1 Využité technologie

4.1.1 Databáze

Pro implementaci databáze byla zvolena **SQLite** databáze, pro její jednoduchost a lehkost na nasazení, vzhledem k malému objemu dat je tato technologie kompletně dostávající.

Pro interakci s databází, byla zvolena technologie **Entity Framework 6 (EF6)**, tato technologie se do programu integrovala pomocí NuGet balíčku a byla zvolena jako ověřená technologie, která značně zlehčuje práci s databází díky překladu LINQ na SQL dotazy. Jedna z klíčových důvodů zvolení této technologie je podpora pro migrace a jejich využití pro automatické vytvoření a nastavení struktury databáze, zejména při nasazení aplikace nebo její aktualizaci, kdy je schopna aktualizovat DB bez ztráty dat. A to jen za pomoci jednoho řádku 4.1

```
public HashTrackDbContext() : base("name=
    DefaultHashTrackSqlLiteConnection")
{
    Database.SetInitializer(new MigrateDatabaseToLatestVersion<
        HashTrackDbContext, Configuration>());
}
```

Listing 4.1: Automatické vytvoření a migrace databáze

4.1.2 IoC

Pro "Inversion of control" byl použit NuGet balíček "Autofac", jeden z nejoblíbenějších IoC kontainerů, byl zvolen pro jakožto ověřená technologie, pro jeho jednoduchost použití a velkou škálu funkcionalit, jako je "Autowire of properties", možnost dynamického skenování pro moduly a assemblies a jejich dynamická registrace. Zde je ukázka použití vlastního atributu pro vlastní dynamickou registraci služeb 4.2.

```
private static IRegistrationBuilder<object, ScanningActivatorData,
    DynamicRegistrationStyle>
    GetStandardServicesPipeline(ContainerBuilder builder, Assembly[]
        assemblies)
{
    return builder.RegisterAssemblyTypes(assemblies)
        .Where(t =>
        {
            var attribute = t.GetCustomAttribute<RegisterServiceAttribute>
                >();
            return attribute != null;
        })
        .As(t =>
        {
            var attribute = t.GetCustomAttribute<RegisterServiceAttribute>
                >();
            if (attribute.ServiceType != null)
                return new[] { attribute.ServiceType };
            return t.GetInterfaces().Where(i => i.Name == $"I{t.Name}");
        });
}
```

Listing 4.2: Dynamická registrace služeb

4.1.3 Uživatelské rozhraní

Pro Model-View-ViewModel (MVVM) byl použit MS NuGet balíček "CommunityToolkit.Mvvm", který byl použit jen pro implementace "ICommand".

```
public class BaseViewModel : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler PropertyChanged;

    protected virtual void OnPropertyChanged([CallerMemberName]
        string propertyName = null)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(
            propertyName));
    }

    protected bool SetField<T>(ref T field, T value, [
        CallerMemberName] string propertyName = null)
    {
        if (EqualityComparer<T>.Default.Equals(field, value)) return
            false;
        field = value;
        OnPropertyChanged(propertyName);
        return true;
    }
}
```

Listing 4.3: Ukázka base ViewModelu

Pro samotné UI, byla použita technologie WPF jelikož, ale Outlook nativně nepodporuje zobrazovat WPF v side panelu rozhraní bylo nutno využít "Windows Form" jako placeholder a s pomocí "ElementHost" v něm zobrazit samotný WPF control.

4.2 Implementace technologií

4.2.1 Interakce s Objektovým Modelem Outlook

Následují ukázky kódu práce s Objektovým Modelem Outlook

4.2.1.1 Přidání kategorie

```
var category = session.Categories[categoryName];  
if (category == null)  
{  
    category = session.Categories.Add(categoryName, (Outlook.  
        OlCategoryColor)categoryColor);  
}  
  
AssignCategoryToItem(item, category);
```

Listing 4.4: Zkrácená ukázka přidání kategorie

4.2.1.2 Práce s „custom properties“

"Custom properties" jsou indexované a tak nám umožňují rychlé vyhledávání v již za-indexovaných artefaktech, k tomuto účelu slouží primárně přidání hashtagů. Zde jsou ukázky obohacení o nalezené hashtagy ?? a obohacení o "artefaktId"4.6

Artefakty v Outlooku nemají žádné perzistentní ID, nejbližší tomuto je EntryID, které se ovšem změnilo přesuneme-li artefakt do jiné složky, čili není spolehlivé, proto artefakt obohacujeme vlastním ID, což nám umožňuje mít referenci na konkrétní artefakt, nalézt jej a při indexingu rozpoznáme, zdali byl již přidán k hashtagu a zamezíme tak duplicitnímu započítání artefaktu.

```
private void EnrichArtefactWithHashTags(Outlook.UserProperties
    properties, string[] hashTags)
{
    var tagsProperty = properties.Find(Constants.CustomProperties.Tags);

    if (tagsProperty is null)
    {
        tagsProperty = properties.Add(
            Constants.CustomProperties.Tags,
            Outlook.OlUserPropertyType.olKeywords,
            true,
            1);
    }

    tagsProperty.Value = hashTags;
}
```

Listing 4.5: Obohacení artefaktu o nalezené hashtagy

```
private Guid EnrichArtefactWithId(Outlook.UserProperties properties)
{
    var property = properties.Find(Constants.CustomProperties.artefactID
        );

    if (property is null)
    {

        property = properties.Add(
            Constants.CustomProperties.artefactID,
            Outlook.OlUserPropertyType.olText,
            true,
            1);
        property.Value = Guid.NewGuid().ToString();
    }

    return Guid.TryParse(property.Value.ToString(), out var id) ? id :
        Guid.Empty;
}
```

Listing 4.6: Obohacení artefaktu o ID

4.2.1.3 Vyhledávání

Zkrácená ukázka spuštění "Advanced Search"

```
public void SearchTags(SearchTagsQueryOptions searchTagsQuery)
{
    string scope = GetScope(searchTagsQuery.Artefacts);
    string filter = GetFilter(searchTagsQuery);
    _application.AdvancedSearch(scope, filter, true, searchTagsQuery.
        EventTag);
}
```

Listing 4.7: Ukázka vyhledávání

4.2.1.4 Ukázka indexingu

Ukázka flow indexingu

```
private void PerformIndexing(Outlook.Search searchResult)
{
    try
    {
        var hashtags = ExtractTagsAndEnrichArtefacts(searchResult);
        var clusteredResults = ClusterHashTags(hashtags).ToHashSet();

        _storage.SaveHashTags(clusteredResults);
        _cache.SetHashTags(clusteredResults.ToList());
        _eventAggregator.FireEvent(Events.HashTagsUpdated);
    }
    catch (System.Exception e)
    {
        _messageService.ShowMessage(e, "Error occurred during indexing");
    }
}
```

Listing 4.8: Ukázka flow Indexingu

4.3 Využité algoritmy

4.3.1 Levenshteinova vzdálenost

Levenshteinova vzdálenost je metrika založená na editační vzdálenosti, používaná k posouzení míry rozdílu mezi dvěma textovými řetězci. Je definována jako minimální počet jednoduchých editačních operací potřebných k převedení jednoho řetězce na druhý. Tyto operace zahrnují:

- Vložení znaku
- Smazání znaku
- Substituce znaku za jiný

$$\text{lev}_{a,b}(i, j) = \begin{cases} \max(i, j) & \text{if } \min(i, j) = 0, \\ \min \begin{cases} \text{lev}_{a,b}(i-1, j) + 1 \\ \text{lev}_{a,b}(i, j-1) + 1 \\ \text{lev}_{a,b}(i-1, j-1) + 1_{(a_i \neq b_j)} \end{cases} & \text{otherwise.} \end{cases}$$

Obrázek 4.1: Levenshteinova vzdálenost [13]

Tato sekce čerpala informace z webů [14] [15]

4.3.1.1 Ilustrace

Pro ilustraci, porovnáváme li slova "mor" a "most", Levenshteinovu vzdálenost mezi těmito slovy vypočítáme následovně:

1. mor $\rightarrow$ most (substituce 'r' za 's')
2. mos $\rightarrow$ most (vložení t na konec slova)

Celkový počet operací je tedy 2, což znamená, že Levenshteinova vzdálenost mezi "mor" a "most" je 2.

Uvažovali bychom slova stejná, pak by Levenshteinova vzdálenost byla 0.

Ukázka porovnání textových řetězců "setting" a "sittmg" pomocí Levenshteinovy vzdálenosti 4.2

		s	e	t	t	i	n	g
	0	1	2	3	4	5	6	7
s	1	0	1	2	3	4	5	6
i	2	1	1	2	3	3	4	5
t	3	2	2	1	2	3	4	5
t	4	3	3	2	1	2	3	4
m	5	4	4	3	2	2	3	4
g	6	5	5	4	3	3	3	3

Obrázek 4.2: Ukázka Levenshteinovy vzdálenosti [16]

4.3.1.2 Využití algoritmu

Levenshteinova vzdálenost má širokou škálu aplikací, včetně:

- Oprava překlepů
- Porovnávání genetických sekvencí v bioinformatice
- Zlepšení automatického doplňování a prediktivního psaní
- Shlukování hashtagů na základě podobnosti

Algoritmus výpočtu normované Levenshteinovy vzdálenosti

Algorithm 1 Výpočet normované Levenshteinovy vzdálenosti:

```
1: function NORMALIZEDLEVENSHTEINDISTANCE( $s, t$ )
2:    $levDistance \leftarrow$  LEVENSHTEINDISTANCE( $s, t$ )
3:    $maxLength \leftarrow$  max(length of  $s$ , length of  $t$ )
4:   if  $maxLength = 0$  then
5:     return 1.0
6:   else
7:     return  $1.0 - (levDistance / maxLength)$ 
8:   end if
9: end function
```

Poznámka: Pseudokód vytvořen na základě implementace NuGet balíčku použitého v projektu z [17]

Algoritmus výpočtu Levenshteinovy vzdálenosti:

Poznámka: Pseudokód čerpán z [18]

Algorithm 2 Výpočet Levenshteinovy vzdálenosti

```
1: function LEVENSHTeINDISTANCE( $s[1..m], t[1..n]$ )
2:    $d[0..m, 0..n]$  ▷ Declare the table of size  $(m + 1) \times (n + 1)$ 
3:   for  $i = 0$  to  $m$  do
4:      $d[i, 0] \leftarrow i$ 
5:   end for
6:   for  $j = 0$  to  $n$  do
7:      $d[0, j] \leftarrow j$ 
8:   end for
9:   for  $i = 1$  to  $m$  do
10:    for  $j = 1$  to  $n$  do
11:      if  $s[i] = t[j]$  then
12:         $cost \leftarrow 0$ 
13:      else
14:         $cost \leftarrow 1$ 
15:      end if
16:       $d[i, j] \leftarrow \min(d[i - 1, j] + 1, d[i, j - 1] + 1, d[i - 1, j - 1] + cost)$ 
17:    end for
18:  end for
19:  return  $d[m, n]$ 
20: end function
```

4.3.1.3 Využití v projektu

Levenshteinův algoritmus byl v projektu využit k realizaci Shlukování hashtagů, na základě podobnosti. Jde o funkcionalitu rozeznání podobných hashtagů a jejich sloučení pro následnou práci. V projektu jsem na Levenshteinův algoritmus, navázal další algoritmy pro přesné vyhodnocování podobnosti.

V hashtagu se může například vyskytovat projekt označený číslem nebo kombinací slov a čísel, pro tento případ není Levenshteinova vzdálenost vhodná, jelikož záměna jedné číslice znamená zneplatnění odkazu na projekt.

Porovnávání řetězce se nejprve rozdělí na textovou a číselnou část, poté se vyhodnotí zdali jsou číselné části shodné, pokud ne pak jsou vyhodnoceny jako neshodné. Pokud se ale číselná část vyhodnotí jako shodná, algoritmus pokračuje dále a vyhodnotí normalizovanou Levensteinovu vzdálenost (1), pokud tato spadá pod nastavenou horní hranici pak jsou hashtagy vyhodnocen jako totožné.

```

public class ClusteringClassifier : IClusteringClassifier
{
    private const double SimilarityThreshold = 0.75;
    private readonly IStringMetric _metric = new Levenstein();

    public bool Classify(string text1, string text2)
    {
        var tag1 = SplitTag(text1);
        var tag2 = SplitTag(text2);

        if (tag1.NumberPart != tag2.NumberPart)
        {
            return false;
        }

        double textualSimilarity = _metric.GetSimilarity(tag1.TextPart, tag2.
            TextPart);
        return textualSimilarity > SimilarityThreshold;
    }

    private static (string TextPart, string NumberPart) SplitTag(string tag)
    {
        var textPartBuilder = new StringBuilder();
        var numberPartBuilder = new StringBuilder();

        foreach (char ch in tag)
        {
            if (char.IsDigit(ch))
                numberPartBuilder.Append(ch);
            else
                textPartBuilder.Append(ch);
        }

        return (textPartBuilder.ToString(), numberPartBuilder.ToString());
    }
}

```

Listing 4.9: Implementace třídy pro klasifikaci textových řetězců

4.4 Aplikace návrhových vzorů

V této je přehled aplikace vybraných návrhových vzorů, jejich využití a příklady implementací.

4.4.0.1 Attribute - Decorator Pattern

Decorator Pattern [12] umožňuje dynamicky přidávat nebo modifikovat chování objektů bez změny jejich kódu, V projektu je využit pro dynamickou Registraci objektů do DI kontejneru (4.11) a určení jejich životního cyklu. Umožňuje registrovat služby přímo v jejich kódu, což zpřehledňuje kód a zjednodušuje údržbu. Při registraci je třeba zvolit pod jakým typem bude služba v DI kontejneru zaregistrována a určit její životní cyklus.

Také se využívá pro registrování speciálních služeb (4.10) pro obsluhu událostí, tato registrace probíhá za pomoci textového řetězce identifikujícího událost a typ pro registraci do DI kontejneru, tyto služby se vždy registrují jako Transientní jelikož po zpracování události již nejsou potřeba a tak je žádoucí aby zanikly. Tohoto je docíleno v kombinaci s "Factory pattern".

```
[RegisterService(LifeCycle.Transient, typeof(
    ISearchCompletedHandlerFactory))]
public class SearchCompletedHandlerFactory :
    ISearchCompletedHandlerFactory
```

Listing 4.10: Využití atributu pro Registrování služby

```
[RegisterHandler(Events.DefaultSearchCompleted, typeof(
    ISearchCompletedHandler))]
public class SearchCompletedHandler : ISearchCompletedHandler
```

Listing 4.11: Využití atributu pro Registrování event handleru

4.4.1 Factory Pattern

Factory Pattern [12] se používá k vytváření objektů bez specifikace konkrétních tříd objektů. Tento návrhový vzor je v projektu využit pro obsluhu událostí (4.12), služba zachytí obecnou událost a na základě jejích dat rozhodne, handler kterého typu má událost obsloužit a ten z DI kontaineru resolvuje.

```
public class SearchCompleteHandlerFactory :
    ISearchCompleteHandlerFactory
{
    private readonly IComponentContext _context;
    public SearchCompleteHandlerFactory(IComponentContext context)
    {
        _context = context;
    }

    public void HandleSearchCompleted(Outlook.Search SearchObject)
    {
        var handler = _context.ResolveKeyed<ISearchCompletedHandler>(
            SearchObject.Tag);
        handler.HandleSearchCompleted(SearchObject);
    }
}
```

Listing 4.12: Využití návrhového vzoru Factory

4.5 Testování

Testování bylo zaměřeno na ověření funkčnosti klíčových funkcí doplňku "HashTrack" pro Outlook. Všechny testy byly prováděny manuálně, přičemž k ověření složitějších funkcionalit byly využity nástroje jako Rider IDE a Datagrip pro inspekci dat v databázi.

4.5.0.0.1 Vyhledávání bylo testováno pro správnou identifikaci a zobrazení artefaktů s konkrétními hashtagy. Testy zahrnovaly vyhledávání jednotlivých i spojených hashtagů, a to i v obsáhlých textech (1000 slov) a různých typech artefaktů.

4.5.0.0.2 Indexování ověřovalo správné identifikování artefaktů obsahujících výskyt libovolného hashtagu a následné přiřazení artefaktů k hashtagům a uložení do databáze.

4.5.0.0.3 Slučování hashtagů testováno automatizované slučování na základě podobnosti hashtagů, včetně případů s překlepy, a ověřovalo správnost zvolených hodnot pro určení podobnosti a správného výsledného spojení.

4.5.0.0.4 Automatická kategorizace testována funkcionality vytváření kategorií pro hashtagy, ověřovalo se správné přiřazování artefaktů do kategorií, a testovalo se chování při změně nastavení kategorizace.

4.5.0.0.5 Persistence se zaměřila na správné ukládání a načítání změněných dat, zejména v kontextu spojených a rozpojených hashtagů. Byly odhaleny některé chyby, které měly za následek ztrátu dat, ty byly následně opraveny.

4.5.0.0.6 Uživatelské rozhraní testováno na odezvu na uživatelské akce a automatickou aktualizaci dat. Při testování byly identifikovány drobné nedostatky v interakci, které byly následně opraveny, s výjimkou jednoho problému s kontextovým menu, který přetrvával, kvůli nativnímu omezení WPF aplikací.

Celkově testování potvrdilo funkčnost implementovaných funkcí s několika drobnými problémy, které byly odstraněny, avšak zůstává prostor pro další zlepšení, zejména v oblasti uživatelského přívětivosti.

4.6 Výzvy a problémy

V rámci vývoji doplňku bylo třeba překonat spousty výzev a nastalo mnoho problémů, v této sekci je popsána hrstka z nich.

4.6.0.0.1 WPF Outlook VSTO neumožňuje nativně vložit do bočního panelu WPF uživatelské rozhraní, tento problém byl vyřešen použitím Windows forms rozhraní a element `ElementHost`, které umožňuje hostovat WPF rozhraní uvnitř Windows forms.

4.6.0.0.2 Dokumentace Velkým problémem v rámci vývoje projektu byla dokumentace VSTO doplňků, jelikož informace v ní byly v některých případech špatné a často se stávalo že odkazované stránky dokumentace na oficiálním webu Microsoftu už nebyly dostupné. Jedním z takových problémů byla dokumentace pro tvorbu DASL queries, která byla velice užitečná ale obsahovala chybu v požadovaném formátu queries. Tento problém bylo obtížné identifikovat, jelikož formát byl podle dokumentace a možností kde se mohla chyba vyskytovat bylo mnoho.

```
@SQL="https://schemas.microsoft.com/mapi/proptag/0x0037001f" = '
    subjectSearchPhrase'
```

Listing 4.13: Formát DASL query v dokumentaci

```
http://schemas.microsoft.com/mapi/proptag/0x0037001f = '
    subjectSearchPhrase'
```

Listing 4.14: Funkční Formát DASL query

4.6.0.0.3 Indexování Jednou z výzev indexování bylo nalézt způsob jakým vyhledávat v těle artefaktů, jelikož tělo artefaktů není indexováno a tak vyhledávající nástroje jako je "MapiFolder search" pomocí metody "Find" nebo "Restrict" nefungují, jelikož prochledávají jen za-indexované pole. Podobně "AdvancedSearch" poskytuje "ci_phrasematch" a "-", ale ani jeden filtr není vhodný k indexování, kde je cílem nalézt všechny artefakty, které obsahují jakékoli slovo ve formátu "#* ". Po dalším průzkumu bylo nalezeno klíčové "ci_startswith", které má hledat slova začínající na konkrétní text nebo znak, což je vhodné, bohužel tento způsob funguje jen pro první slovo těla, jelikož není indexováno. Nakonec bylo nalezeno slovo "like", které je schopno prohledat celé tělo zprávy a tak nalézt jakýkoli artefakt obsahující výskyt hashtagu v těle.

4.6.0.0.4 COM VSTO využívá "Component object model"(COM) pro komunikaci s Outlookem, tato technologie přináší své komplikace při tvorbě aplikace. Absence unifikovaného rozhraní, objekty jako MailItem, AppointmentItem, ContactItem, TaskItem nabízí mnoho totožných metod a vlastností, ale nemají žádné společné rozhraní což komplikuje hromadnou práci s těmito objekty. Technologie COM také nebyla nevržena pro práci v asynchroním kontextu a tak jej nepodporuje, což stěžuje tvorbu responzivních aplikací, jelikož se vše zpracovává ve stejném vlákne jako uživatelské rozhraní a výsledkem je neresponzivní UI. Toto bylo řešeno prováděním dlouho běžících operací jako je zpracovávání dat pro indexování v nově vytvořeném vlákne v rámci doplňku.

4.6.0.0.5 Instalace Velkým praktickým problémem je nezprovoznění funkční instalace, ačkoliv instalační soubor je nastaven publikuje se, po provedení instalace se doplněk začne spouštět, ale nespustí se správně a tak není jej možno používat. Tento problém nebyl vyřešen a prozatím je zapotřebí dělat "instalaci" za pomoci programu "Visual studio" ze zdrojového kódu aplikace. Je třeba aplikaci sestavit, což automaticky zavede aplikaci do registru a připraví ji k používání.

4.6.0.0.6 Další Mezi další řešené problémy patří, propojování hashtagů, udržení reference mezi spojenými hashtagy pro aktuálnost dat a vytvoření této reference při načítání hashtagu z databáze. Shlukování, řešení podobnosti hashtagů a určení pravidel, kdy se mohou a kdy se nemohou hashtagy spojit. A další.

Kapitola 5

Poznatky a závěr

5.1 Poznatky

Z projektu je jasné, že doplňky pro Outlook mohou být velice silné nástroje, které zjednoduší práci a zlepší produktivitu. Ale v rámci implementace projektu byly odhaleny slabé stránky VSTO/COM doplňků. Zejména kvůli využívané COM technologie, která je omezující. A to jak v neexistující nativní podpoře asynchronního programování, které musí být obcházeno aby byla aplikace responzivní, tak i ve stáří technologie, což sebou přineslo nečekané výzvy. Pochopitelně bylo za potřebné používat kompatibilní technologie, které nemusí být vždy stále neudržované, což přináší své nevýhody. Překvapivé bylo, že naproti očekávání že bude technologie vyzrálá se však objevilo mnoho problémů a to jak se zastaralou dokumentací, tak i s chybami v dokumentaci. Z čehož je jasné, že držet se udržovaných, ale vyzrálých technologií je výhodné.

Pro budoucí vývoj Outlook doplňku bych zvažoval modernější typ doplňku jako je Web add-in.

5.2 Závěr

V této práci se rozebrali technologie a možnosti pro tvorbu doplňků pro MS Outlook, detail Objektového Modelu Outlook, který tyto technologie využívají a výsledkem je i funkční doplněk pro vyhledávání, správu a manipulaci s hashtagy v aplikaci MS Outlook, který by mohl být dále upravován a vylepšen. Body zlepšení by měli být jak v ohledech uživatelského rozhraní, uživatelské zkušenosti tak i ve funkcionalitách kde by mohly být přidány další funkcionality zlehčující uživatelům jejich práci. Hlavním bodem ke zlepšení doplňku je instalace, jelikož proces je komplikovaný a není uživatelsky přívětivý.

Literatura

1. *Microsoft AppSource* [online]. 2024. [cit. 2024-04-16]. Dostupné z: <https://appsource.microsoft.com/en-us/marketplace/>.
2. *Architektura doplňků VSTO* [online]. 2023. [cit. 2024-04-16]. Dostupné z: <https://learn.microsoft.com/cs-cz/visualstudio/vsto/architecture-of-vsto-add-ins?view=vs-2022>.
3. *Office Add-in Architecture* [online]. 2024. [cit. 2024-04-16]. Dostupné z: <https://learn.microsoft.com/cs-cz/visualstudio/vsto/media/office07addin.png?view=vs-2017%5C&viewFallbackFrom=vs-2022>.
4. *Přehled rozhraní .NET Framework* [online]. [cit. 2024-04-16]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/framework/get-started/overview>.
5. *Přehled objektového modelu Outlooku* [online]. 2023. [cit. 2024-04-16]. Dostupné z: <https://learn.microsoft.com/cs-cz/visualstudio/vsto/outlook-object-model-overview?view=vs-2022>.
6. *Application Interface* [online]. 2024. [cit. 2024-04-17]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/api/microsoft.office.interop.outlook.application?view=outlook-pia>.
7. *MailItem Interface* [online]. 2024. [cit. 2024-04-17]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/api/microsoft.office.interop.outlook.mailitem?view=outlook-pia>.
8. *AppointmentItem Interface* [online]. 2024. [cit. 2024-04-17]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/api/microsoft.office.interop.outlook.appointmentitem?view=outlook-pia>.
9. *ContactItem Interface* [online]. 2024. [cit. 2024-04-17]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/api/microsoft.office.interop.outlook.contactitem?view=outlook-pia>.

10. *TaskItem Interface* [online]. 2024. [cit. 2024-04-17]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/api/microsoft.office.interop.outlook.taskitem?view=outlook-pia>.
11. PFLEEGER, Shari; ATLEE, Joanne. *Software Engineering: Theory and Practice: Prentice Hal*. 4. vyd. Prentice Hall, 2009. ISBN 0136061699.
12. ERICH, Gamma; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. *Design Patterns: Elements of Reusable Object-Oriented Software*. 1. vyd. Addison-Wesley Professional, 1994. ISBN 978-0201633610.
13. *Levenshtein Algorithm - example* [online]. 2017. [cit. 2024-04-17]. Dostupné z: <https://www.cuelogic.com/wp-content/uploads/2021/06/Maths.jpg>.
14. *Introduction to Levenshtein distance* [online]. 2024. [cit. 2024-04-17]. Dostupné z: <https://www.geeksforgeeks.org/introduction-to-levenshtein-distance/>.
15. *Levenshteinova vzdálenost - Algoritmy.net* [online]. 2016. [cit. 2024-04-17]. Dostupné z: <https://www.algoritmy.net/article/1699/Levenshteinova-vzdalenost>.
16. *Levenshtein Distance - example* [online]. 2020. [cit. 2024-04-17]. Dostupné z: <https://blog.paperspace.com/content/images/2020/02/Fig13.jpg>.
17. HEYENRATH, Stef. *Levenshtein metric implementation by SimMetrics.Net* [online]. 2017. [cit. 2024-04-17]. Dostupné z: <https://github.com/StefH/SimMetrics.Net/blob/master/src/SimMetrics.Net/Metric/Levenshtein.cs>.
18. R, Tommy. *Levenshtein Pseudocode* [online]. 2008. [cit. 2024-04-17]. Dostupné z: <https://gist.github.com/tommy/14631>.

Příloha A

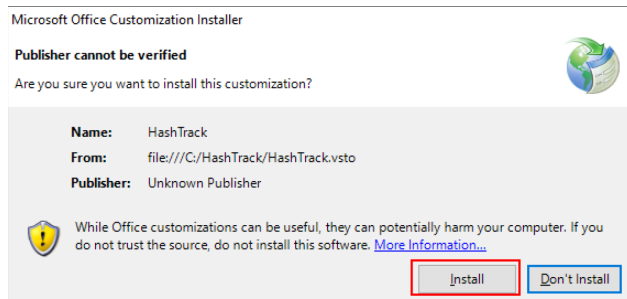
Manuál

A.1 Instalační manuál

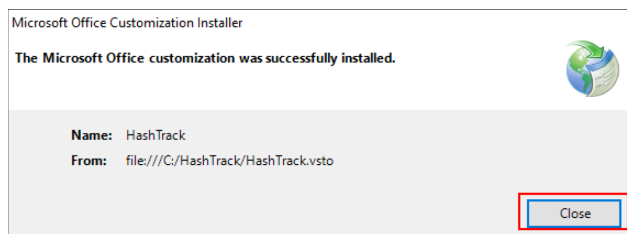
1. Otevřete instalační složku.
2. Spustíte soubor "Setup.exe".

Application Files	File folder	
HashTrack.vsto	VSTO Deployment...	6 KB
setup.exe	Application	957 KB

3. Klikněte na **"Install"** a vyčkejte dokončení instalace.



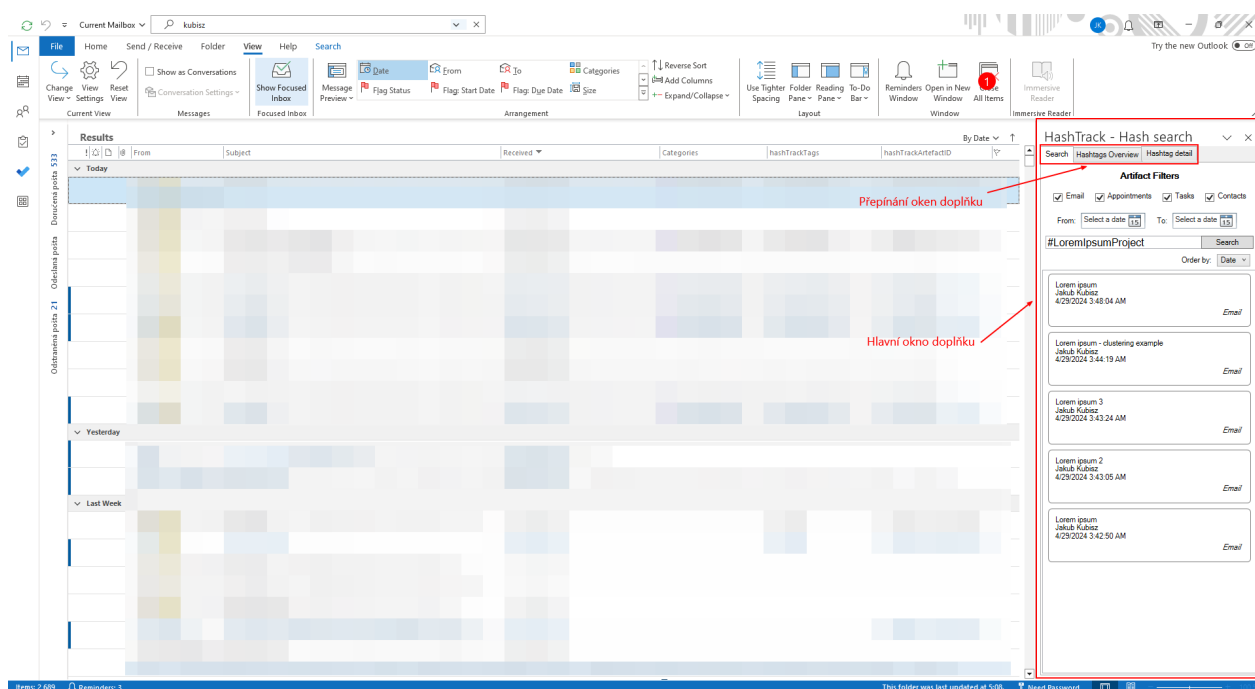
4. Hotovo, klikněte na **"Close"**.



A.2 Uživatelský manuál

Doplňek "HashTrack"slouží ke spravování a vyhledávání artefaktu v aplikaci Outlook za použití hashtagů. Artefakty se rozumí e-maily, úkoly, shůzky a kontakty. HashTrack umožňuje vyhledávání artefaktů s výskytem konkrétního hashtagu, jejich zobrazení a zobrazení detailu každého artefaktu. Dále nabízí přehled všech nalezených hashtagů v artefaktech a umožňuje upravovat nastavení těchto hashtagů.

Jak je zvýrazněno v A.1, uživatelské rozhraní doplňku HashTrack je v pravé části aplikace Outlook a skládá se z několika záložek. Každá záložka nabízí jiné funkcionality, níže naleznete návod jak pracovat s každou záložkou.



Obrázek A.1: Outlook s doplňkem hashtrack

A.2.1 Vyhledávání

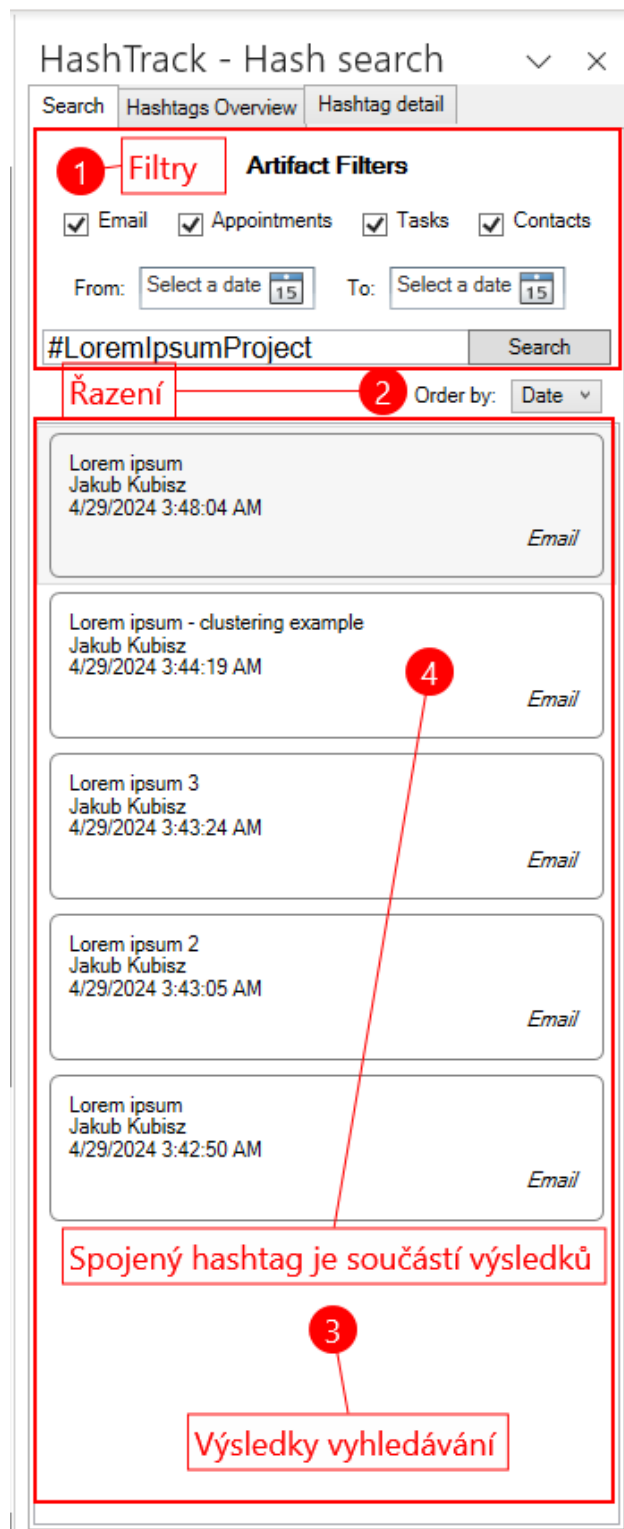
Vyhledávání hashtagu je možno provádět na záložce vyhledávání "Search" A.2. Tato část je určena k vyhledávání artefaktů s použitím libovolných hashtagů, a umožňuje vyhledávání filtrovat na základě typu artefaktu, a datumu.

Filtrování se nastavuje v sekci 1. "Filtry". Vždy je nutné vyplnit celý vyhledávaný hashtag a vybrat alespoň jeden typ artefaktu, který chceme hledat, pokud datum zůstane nevyplněné tak se prohledává celá historie schránky.

Je možno vyhledané výsledky seřadit pomocí sekce 2. "Řazení".

Seřazené a vyfiltrované výsledky vyhledávání se zobrazují v sekci 3. "Výsledky vyhledávání", zde můžeme vidět všechny artefakty, které vyhledávání našlo. Je také možnost otevřít detail artefaktu a to dvojitým poklepáním na artefakt.

V ukázkovém obrázku je také sekce 4. "Spojený hashtag je součástí výsledků", toto je ukázka toho, že v rámci vyhledávání hashtagu se zobrazují aji jeho spojené hashtagy, více informacím ke spojení hashtagů naleznete v sekci "Detail hashtagu" (A.2.3)



Obrázek A.2: záložka Vyhledávání

A.2.2 Přehled hashtagů

Přehled hashtagů se nachází v záložce přehledu hashtagů "Hashtags Overview" A.3. Zde jsou zobrazeny všechny známé hashtagy, které byly nalezeny pomocí za-indexování schránky. Indexování je process prohledání schránky, k nalezení výskytu hashtagů, a označení jejich artefaktů. Tento process se provádí automaticky po spuštění aplikace a také se spouští automaticky v nastaveném časovém intervalu. Automatický indexing, ale vždy indexuje jen nové zprávy (od posledního indexingu), v případě, že je žádoucí schránku re-indexovat, ať už z důvodů vymazání záznamů nebo je účel za-indexovat i staré zprávy, je zde možnost manuálního spuštění indexování popsána níže.

Manuální indexování se provádí v sekci "Re-indexování a nastavení data", je možné určit rozsah indexování pomocí výběru rozsahu dat, mezi kterými se mají artefakty indexovat, pokud datum nebude vyplněn použije se výchozí rozsah indexování. Po nastavení vhodného rozsahu se dvojitým stisknutím tlačítka "Re-index" spustí process indexování, během procesu je možno nadále používat aplikaci Outlook aji doplněk hashtrack, není však možno měnit nastavení Hashtagu během indexování.

Je také možnost filtrovat a řadit zobrazené výsledky indexování, toto se provádí v sekci "Řazení a filtrování", vyplnění polí jen filtruje zobrazené výsledky a neovlivňuje stav ani uložené data indexování. Je možno filtrovat podle textových částí hashtagu nebo podle počtu nalezených artefaktů u hashtagu pomocí nastavení minimální-ho a nebo i maximální-ho počtu artefaktů u hashtagu.

Seřazené hashtagy nalezené indexováním a vyfiltrované vyplněnými filtry se zobrazují v sekci "Nalezené hashtagy" a to v následujícím formátu "#Hashtag - počet artefaktů - počet spojených hashtagů - počet rozpojených hashtagů" viz. ukázka 1. "Formát", počet spojených hashtagů je obarven do zelena a zobrazuje se jen pokud existují spojené hashtagy, počet rozpojených hashtagů je obarven do červena a zobrazuje se jen pokud existují rozpojené hashtagy.

HashTrack - Hash search

SearchHashtags OverviewHashtag detail

From: Select a date15To: Select a date15

Re-indexování a nastavení dataRe-index

ore

Num usages From: 0To: 0

Řazení a filtrováníOrder by: Most used

#LoremIpsumProject - 4 - 1 - 11

#oremlpsumProject - 1

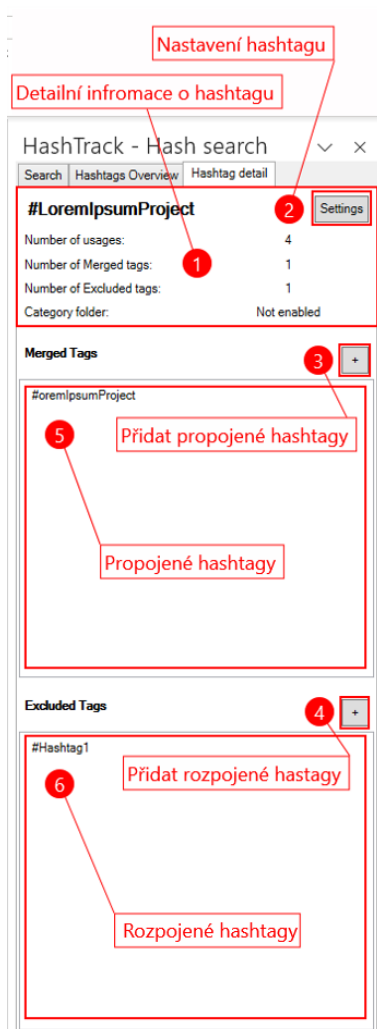
Nalezené hashtagy
- po filtrování

Formát:
#hashtag
- počet artefaktů
- počet propojených hashtagů
- počet rozpojených hashtagů

Obrázek A.3: Přehled zaindexovaných hashtagů

A.2.3 Detail Hashtagu

Detail hashtagu se nachází v záložce detailu hashtagu "Hashtag detail" A.4 Detail hashtagu poskytuje detailní informace o hashtagu v sekci 1. "Detailní informace o hashtagu". Dale obsahuje seznam připojených hashtagu v sekci "Propojené hashtagy" s tlačítkem "+" (3. "Přidat propojený hashtag") pro připojení hashtagů a seznam rozpojených hashtagů v sekci "Rozpojené hashtagy" s tlačítkem "+" (4. "Přidat rozpojený hashtag") pro připojení hashtagů. Pro další nastavení hashtagu je třeba otevřít vyskakovací okno nastavení, kliknutím na tlačítko "Settings" označeného 2. "Nastavení hashtagu" více informací k vyskakovacímu oknu nastavení je uvedeno níže.

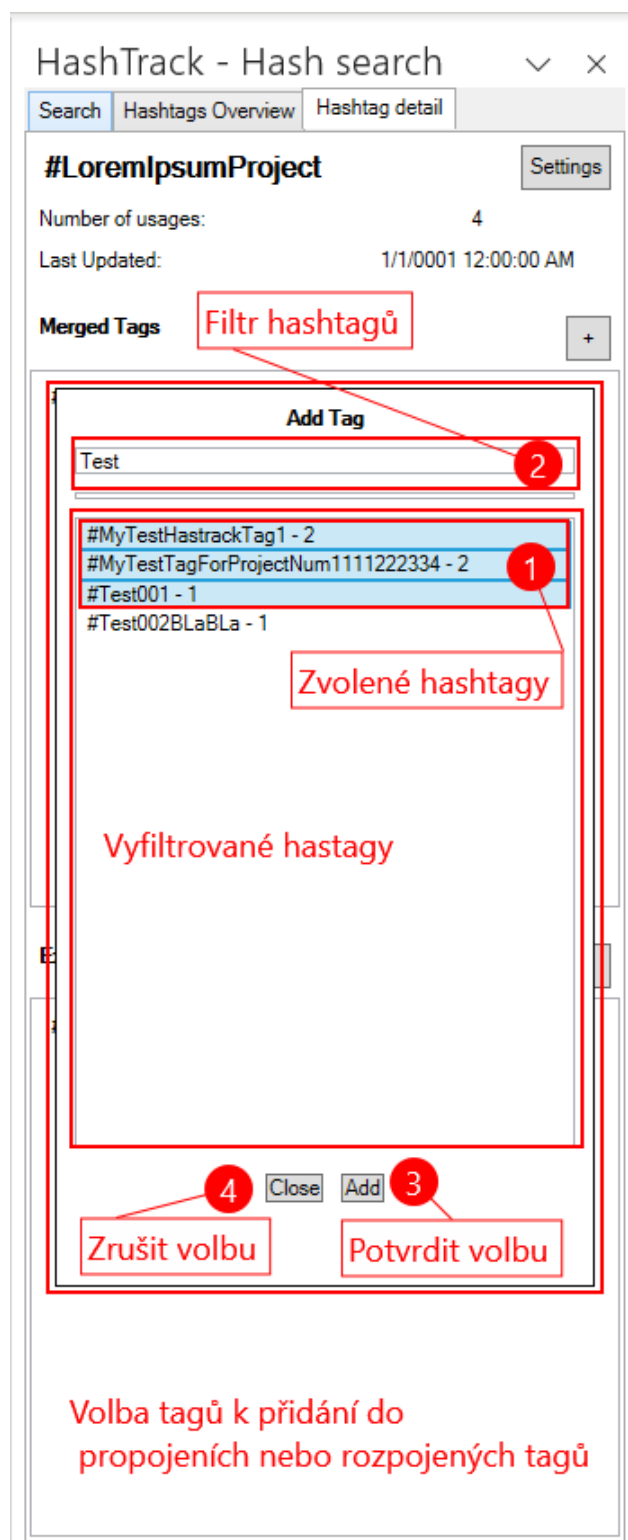


Obrázek A.4: Detailní pohled

A.2.3.1 Vyskakovací okno pro výběr hashtagu

Po kliku na tlačítko + pro rozpojení nebo spojení hashtagů se zobrazí vyskakovací okno přidání hashtagu A.5 s textovým filtrem na podslova hashtagu, v sekci 2. "Filtr hashtagů" a seznam vyfiltrovaných hashtagů v sekci "Vyfiltrované hashtagy". Ze seznamu je možno zvolit jeden nebo více hashtagů k přidání, jak je možno vidět v 1. "zvolené hashtagy". Výběr se potvrdí tlačítkem "Add" označeném jako 3. "Potvrdit volbu" a nebo je možno vyskakovací okno zavřít bez zložení změn pomocí tlačítka "Close" označeného 4. "Zrušit volbu".

Nastavení hashtagu zahrnují funkci spojení více hashtagů, tato funkcionalita se hodí pro případy kdy je žádoucí spojit 2 témata nebo hashtagy, ve kterých se vyskytnul překlep. Překlepy jsou automaticky detekovány na základě podobnosti hashtagů a v takovém případě jsou hashtagy spojeny automaticky. Dojde-li k nesprávnému spojení nebo chceme-li zvrátit spojení dvou témat přichází na řadu funkcionalita rozpojení. Funkcionalita rozpojení je velice podobná spojení hashtagů, ale naopak zajišťuje aby s hashtagy bylo zacházeno jako s rozdílnými hashtagy, to znamená, že již nebudou kontrolovány na podobnost a nebudou automaticky spojeny.

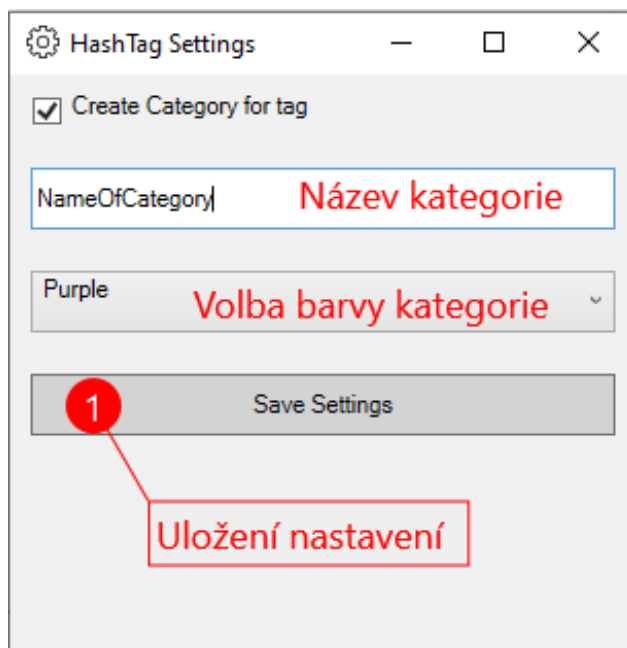


Obrázek A.5: Přidání nového hashtagu

A.2.3.2 Vyskakovací okno nastavení hashtagu

Vyskakovací okno nastavení A.6, obsahuje další nastavení hashtagu, které nejsou tak často používány, je zde možnost zapnout automatické vytváření Outlook kategorií. Pokud je tato možnost na hashtagu zapnuta tak všechny artefakty obsahující tento hashtag (a také jeho spojené hashtagy) budou automaticky přidány do kategorie hashtagu. Toto umožňuje jasné zviditelnění artefaktu v uživatelském rozhraní outlook a otevírá nové možnosti filtrace artefaktů pomocí těchto kategorií.

Ve vyskakovacím okně nastavení hashtagu "HashTag Settings" A.6, je možnost zapnout automatické vytváření a přiřazování kategorií, kliknutím na přepínač "Create Category for tag", po povolení této možnosti se nastavení automaticky předvyplní výchozím nastavením, což znamená že název Název kategorie je nastaven na hashtag, zde je možnost vybrat vlastní název kategorie a ze seznamu zvolit libovolnou barvu kategorie, toto nastavení se potvrdí tlačítkem "Save Settings" označeném 1. "Uložení nastavení".



Obrázek A.6: Nastavení hashtagů